ARGUS

Exam Supervision Network Monitoring System

Feature Documentation Report

Repository: `DevNexus09/Argus-ExamSupervisonSystem`

FEATURES COVERED

Feature 1 — Secure Tunnel (Hybrid Cryptography: RSA + RC4)
Feature 2 — Encrypted Traffic Analysis (Shannon Entropy / VPN Detection)
Feature 3 — Deep Packet Inspection (TLS SNI + Aho-Corasick)
Feature 4 — Time Synchronization & Clock Tamper Detection (Cristian's Algorithm)
Feature 5 — Huffman Coding Compression for Violation Messages

Feature 1 · **Secure Communication Tunnel**

Hybrid Cryptography — RSA + RC4 Stream Cipher

Files	protocol.cpp · protocol.h · echo_student.cpp · echo_supervisor.cpp
Algorithm	Hybrid Cryptography (RSA + RC4-style XOR stream cipher)
Applies to	Student Agent ↔ Supervisor Server handshake & all payload traffic

1. Algorithm Overview

This feature implements Hybrid Cryptography — combining two complementary encryption methods to achieve both security and performance:

- **RSA (Asymmetric Encryption)** — used exclusively during the initial connection handshake to safely exchange a session key without transmitting it in plaintext.
- **RC4-style Stream Cipher (Symmetric Encryption)** — used for all subsequent message payloads after handshake; provides fast, continuous encryption via XOR with a keystream.

2. What the Feature Does

The feature establishes a secure tunnel between the Student Agent and the Supervisor Server. When a student connects, a handshake negotiates a shared session key using RSA. After that, every message payload is encrypted with the RC4-derived keystream, preventing eavesdroppers from reading or modifying supervision data.

3. Why This Feature Is Needed

Argus transmits sensitive supervision signals over the network:

- Violation reports (websites/domains accessed)
- Heartbeat signals (student agent is alive)
- Tamper alerts (agent terminated or disturbed)

Without encryption, an attacker on the same network could read monitoring messages (privacy breach), alter messages (hide or fabricate violations), or inject forged messages (impersonate a student). This feature provides:

- **Confidentiality** — data remains private in transit.
- **Integrity** — casual message modification becomes infeasible.
- **Trust** — authenticated channel between Student Agent and Supervisor.

4. Algorithm & Formulas

A) RSA Key Generation (Server Side)

The server selects two prime numbers P and Q and derives the full RSA keypair:

$$N = P \times Q \qquad \text{(modulus)}$$
$$\Phi = (P - 1) \times (Q - 1) \qquad \text{(Euler's totient)}$$
$$\gcd(E, \Phi) = 1 \qquad \text{(public exponent E)}$$
$$(E \times D) \bmod \Phi = 1 \qquad \text{(private exponent D — modular inverse of E)}$$

The server publishes (N, E) and keeps D secret.

B) RSA Encryption (Student Side)

The student generates a random 16-character session key, then encrypts each character m with the server's public key:

$$\text{Cipher} = (m \wedge E) \bmod N$$

C) RSA Decryption (Server Side)

The server recovers each plaintext character from the ciphertext c using the private key D:

$$\text{Message} = (c \wedge D) \bmod N$$

D) RC4-style Stream Cipher (Post-Handshake)

After both sides share the session key, all payload bytes are XOR-encrypted with a pseudo-random keystream generated from the key:

$$\text{EncryptedByte} = \text{PlainByte} \ \text{XOR} \ \text{KeystreamByte}$$
$$\text{PlainByte} = \text{EncryptedByte} \ \text{XOR} \ \text{KeystreamByte} \quad \text{(same operation — XOR is reversible)}$$

5. Codebase Implementation

5.1 Common Crypto Layer — protocol.cpp

RC4 Logic

- Function: RC4_Logic(char* data, int length, const string& key)
- Initializes S[256], applies key-scheduling, generates keystream, XORs each payload byte.
- Wrapper: SecureEncrypt() / SecureDecrypt() both call the same RC4_Logic function.

RSA — Modular Exponentiation

The Power(base, exp, mod) function implements fast square-and-multiply to compute both (m^E) mod N and (c^D) mod N efficiently.

RSA Key Generation — GenerateRSAKeys(n, e, d)

```
// Primes chosen from [100, 500]
n   = p * q
phi = (p - 1) * (q - 1)
e   — starting from 3, incremented until gcd(e, phi) == 1
d   = ModInverse(e, phi)
```

5.2 Payload-Only Encryption in serialize() / deserialize()

Argus encrypts only the payload (data field), not the message header, so routing metadata remains readable while sensitive content is protected.

serialize() — Encryption

```
// Writes header + payload to buffer, then:
SecureEncrypt(buffer + offset, dataSize, key);
```

deserialize() — Decryption

```
// Reads header, copies payload, then:
SecureDecrypt(msg->data, dataSize, key);
// If msgViolationCompressed → decompress after decrypt
```

5.3 Student Handshake — echo_student.cpp · performHandshake()

- Send msgHandshakeInit → signals intent for secure connection.
- Receive msgHandshakeKey → extract serverN and serverE.
- Generate 16-char random session key (currentSessionKey).
- Encrypt each character: encryptedChar = Power((long long)c, serverE, serverN)
- Send msgHandshakeResponse with encrypted session key payload.
- Receive msgACK encrypted with the new session key — confirms sync.

5.4 Supervisor Handshake — echo_supervisor.cpp

On receiving msgHandshakeInit

```
GenerateRSAKeys(n, e, d);
clientContext[sd].privateKeyD = d;
clientContext[sd].publicKeyN  = n;
// Send msgHandshakeKey with payload (n, e)
```

On receiving msgHandshakeResponse

```
// Decrypt each value from payload:
decryptedChar = (char) Power(encryptedChar, d, n);
// Store recovered session key
clientContext[sd].sessionKey           = recoveredKey;
clientContext[sd].isHandshakeComplete  = true;
// Send ACK encrypted with recoveredKey
serialize(ackMsg, ackBuffer, recoveredKey);
```

SUMMARY *Argus uses RSA to securely exchange a per-session key, then switches to a fast RC4-style XOR cipher for all subsequent payloads — giving both strong key establishment and efficient runtime encryption.*

Feature 2 · Encrypted Traffic Analysis

VPN / Encrypted Tunnel Detection via Shannon Entropy

File	Code/Student/Src/echo_student.cpp
Algorithm	Shannon Entropy (Information Theory)
Applies to	Student Agent network sniffer pipeline

1. Algorithm Overview

Shannon Entropy measures the randomness of data. For byte-level data the output ranges from 0.0 (fully predictable) to 8.0 bits/byte (perfectly random):

- Low entropy → plain text, HTTP headers, JSON (predictable patterns)
- High entropy → encrypted traffic, VPN tunnels, compressed data (random-looking)

2. What the Feature Does

A behavior-based VPN / encrypted tunnel detector. Instead of attempting to decrypt packets, the Student Agent:

- Captures the raw payload of TCP/UDP packets via pcap.
- Computes Shannon Entropy of each payload.
- Groups packets into flows keyed by destination IP + port.
- Maintains a rolling average entropy for the first packets of each flow.
- Triggers a violation alert if the sustained average exceeds the threshold.

3. Why This Feature Is Needed

Students may use VPNs, encrypted tunnels, or encrypted proxy tools to hide their traffic. Deep Packet Inspection (keyword scanning inside payloads) fails because the payload is opaque ciphertext.

VPN traffic is encrypted and therefore appears highly random to an observer. Shannon Entropy quantifies this randomness, allowing Argus to distinguish VPN tunnels from ordinary unencrypted traffic (e.g. plain HTTP), which has much lower entropy — without ever decrypting the traffic.

4. Algorithm & Formula

Shannon Entropy Formula

For a payload buffer, count the frequency of each byte value (0–255), compute probabilities, and apply the entropy sum:

$$H(X) = -\Sigma\ p(x) \cdot \log_2(\,p(x)\,) \qquad \text{for } x \in \{0 \ldots 255\}$$

$$\text{where } p(x) = \text{count}(x) / \text{payload_length}$$

Interpretation: if only a few byte values repeat, entropy is low (predictable); if all values appear roughly equally, entropy approaches 8.0 (random / encrypted).

Detection Workflow

```
[Packet captured by Student Agent (pcap)]
           |
           v
[Extract IP header + TCP/UDP header]
           |
           v
[Extract payload bytes + payload_len]
           |
           v
[Compute Shannon Entropy(payload)]
           |
           v
[Build Flow Key = dest_ip : dest_port]
           |
           v
[Update flow stats: packetCount, totalEntropy]
           |
           v
[If packetCount >= 10]
           |
           v
[avgEntropy = totalEntropy / packetCount]
           |
           v
[If avgEntropy > 7.8  AND  not alerted]
           |
           v
[Trigger "Potential VPN/Encrypted Tunnel" violation --> send to supervisor]
```

5. Codebase Implementation

A) Entropy Function — calculateShannonEntropy()

```
double calculateShannonEntropy(const u_char* data, int length) {
    if (length <= 0) return 0.0;
    int counts[256] = {0};
    for (int i = 0; i < length; ++i)
        counts[data[i]]++;

    double entropy = 0.0;
    for (int i = 0; i < 256; ++i) {
        if (counts[i] > 0) {
            double p = (double)counts[i] / length;
            entropy -= p * log2(p);
        }
```

```
    }
    return entropy;
}
```

B) Flow Tracking — FlowData struct

```
struct FlowData {
    int    packetCount  = 0;
    double totalEntropy = 0.0;
    bool   alerted      = false;
};

map<string, FlowData> activeFlows;   // key = dest_ip:dest_port
```

C) Detection Logic — packet_handler()

```
string flowKey = string(dest_ip) + ":" + to_string(dport);
double entropy = calculateShannonEntropy(payload, payload_len);
FlowData& flow = activeFlows[flowKey];

if (!flow.alerted && flow.packetCount < 20) {
    flow.packetCount++;
    flow.totalEntropy += entropy;

    if (flow.packetCount >= 10) {
        double avgEntropy = flow.totalEntropy / flow.packetCount;
        if (avgEntropy > 7.8) {
            flow.alerted = true;
            string alertMsg = "Potential VPN/Encrypted Tunnel Detected. "
                            + "Destination: " + string(dest_ip)
                            + ". Sustained Entropy: " + to_string(avgEntropy);
            // Create and send violation message to supervisor
            sendMessage(msg);
        }
    }
}
```

Note: Argus averages over the first 10–20 packets and uses a sustained threshold of 7.8 (just below the maximum of 8.0) to reduce false positives from occasional high-entropy payloads or compressed data.

SUMMARY *Argus detects VPN and encrypted tunnels by measuring how random payload bytes look, using Shannon Entropy. The Student Agent averages entropy across a flow and reports a violation when sustained randomness indicates encryption — without ever decrypting the traffic.*

Feature 3 · **Deep Packet Inspection**

TLS SNI Extraction + Aho-Corasick Payload Scanning

Files	trie.h · trie.cpp · echo_student.cpp
Algorithm	Aho-Corasick multi-pattern search + TLS SNI extraction
Applies to	Student Agent HTTPS traffic (TCP port 443)

1. Algorithm Overview

This feature combines three complementary techniques:

- **Deep Packet Inspection (DPI)** — inspects the application payload, not just IP/port headers.
- **TLS SNI Extraction** — reads the target domain name from the TLS ClientHello, which appears in plaintext before full encryption begins.
- **Aho-Corasick Multi-Pattern Search** — scans the payload against many blacklisted keywords simultaneously in O(n) time — fast enough for real-time packet analysis.

2. What the Feature Does

A real-time packet scanning system in the Student Agent that inspects all HTTPS traffic (TCP port 443). For each captured packet it:

Extracts the SNI domain from the TLS ClientHello and compares it against the whitelist.

Scans the raw payload bytes for blacklisted keywords using Aho-Corasick.

Sends a violation message to the supervisor if either check fails.

3. Why This Feature Is Needed

DNS-only filtering (UDP port 53) can be bypassed by students using:

- DNS-over-HTTPS (DoH)
- Manual hosts file edits
- Direct IP access or custom resolvers

Even when DNS is bypassed, the browser still opens a TCP connection and sends a TLS ClientHello — which contains the destination domain (SNI) in plaintext before encryption fully starts. DPI intercepts this, making Argus significantly harder to circumvent with local tricks.

4. Algorithm Logic

Aho-Corasick — Key Concepts

- **Trie Build** — all blacklisted keywords are inserted into a Trie.
- **Failure Links** — BFS assigns each Trie node a failure pointer so that on a character mismatch the search jumps to the longest proper suffix that is still a Trie prefix — no backtracking needed.
- **Search** — one linear pass over the payload: O(n) where n = payload length, regardless of the number of keywords.

DPI Workflow

```
[pcap captures packet]
        |
        v
[Parse Ethernet + IP header]
        |
        v
[TCP packet: parse TCP header]
        |
        v
[Compute payload pointer + payload_len]
        |
        v
[port 443 traffic?]
    |               |
    v               v
[SNI extraction]  [Aho-Corasick scan]
    |               |
    v               v
[Whitelist check] [Blacklist keyword hit?]
         \           /
          v         v
   [Violation --> msgViolation --> Supervisor]
```

5. Codebase Implementation

A) Trie + Failure Links — trie.h / trie.cpp

TrieNode Structure

```
struct TrieNode {
    TrieNode* children[alphabetSize];
    bool      isEnd;      // true = end of a blacklisted keyword
    TrieNode* fail;       // Aho-Corasick failure / fallback pointer
};
```

BuildFailureLinks() — BFS construction of failure pointers

```
void BuildFailureLinks(Trie* trie) {
    trie->root->fail = nullptr;
```

```
    queue<TrieNode*> q;
    // Root's children point back to root on failure
    for (int i = 0; i < alphabetSize; i++) {
        if (trie->root->children[i]) {
            trie->root->children[i]->fail = trie->root;
            q.push(trie->root->children[i]);
        }
    }
    while (!q.empty()) {
        TrieNode* current = q.front(); q.pop();
        for (int i = 0; i < alphabetSize; i++) {
            if (current->children[i]) {
                TrieNode* child    = current->children[i];
                TrieNode* failNode = current->fail;
                while (failNode && !failNode->children[i])
                    failNode = failNode->fail;
                child->fail = failNode ? failNode->children[i] : trie->root;
                if (child->fail->isEnd) child->isEnd = true;
                q.push(child);
            }
        }
    }
}
```

AhoCorasickSearch() — O(n) single-pass scan

```
bool AhoCorasickSearch(Trie* trie, const char* text, int len) {
    TrieNode* current = trie->root;
    for (int i = 0; i < len; i++) {
        int index = CharToIndex(text[i]);
        if (index == -1) continue;
        while (current && !current->children[index])
            current = current->fail;
        current = current ? current->children[index] : trie->root;
        if (current->isEnd) return true;   // keyword hit
    }
    return false;
}
```

B) TLS SNI + DPI Enforcement — echo_student.cpp

SNI Extraction — extractSNI(payload, payload_len)

- Checks if payload starts with TLS handshake bytes (0x16, 0x03 …)
- Navigates ClientHello extension fields.
- Locates extension type 0x0000 (SNI) and reads the domain string.

Packet Header Stripping — packet_handler()

```
payload     = packet + ip_offset + ip_len + tcp_len;
payload_len = caplen - (ip_offset + ip_len + tcp_len);
```

HTTPS DPI Logic — port 443 enforcement

```
if (iph->ip_p == IPPROTO_TCP && (dport == 443 || sport == 443)) {
    if (dport == 443) {
        string sni = extractSNI(payload, payload_len);
        if (!sni.empty() && !isAllowed(sni)) {
            // Build and send SNI violation message
            sendMessage(msg);
        }
    }
    if (AhoCorasickSearch(&blacklistTrie, (const char*)payload, payload_len)) {
        string alertMsg = "DPI Blocked Content. Dest IP: " + string(dest_ip);
        sendMessage(msg);
    }
}
```

SUMMARY *Argus uses DPI on HTTPS traffic to catch students who bypass DNS filtering. TLS SNI extraction reveals the target domain in plaintext before encryption, while Aho-Corasick scans raw payloads for blacklisted content in a single O(n) pass — both feed violations directly to the supervisor.*

Feature 4 · **Trusted Time Sync & Clock Tamper Detection**

Cristian's Algorithm + Monotonic Clock Comparison

Files	protocol.h · echo_student.cpp · echo_supervisor.cpp
Algorithm	Cristian's Algorithm (network time sync) + monotonic clock drift detection
Applies to	Student Agent startup + continuous runtime tamper monitoring

1. Algorithm Overview

Cristian's Algorithm estimates the true server time by measuring round-trip latency and compensating for one-way network delay. Argus supplements this with a monotonic clock (steady_clock — a hardware timer that cannot be set by the OS user) for tamper detection.

2. What the Feature Does

At startup — after the secure handshake — the Student Agent:

- Sends 3 time-sync requests to the Supervisor.
- Records the RTT for each attempt.
- Selects the attempt with the lowest RTT (best network conditions).
- Computes a clockOffset and stores it.
- Uses trustedTime = time(0) + clockOffset for all future message timestamps.

During runtime — continuously:

- Tracks elapsed time via steady_clock (monotonic, cannot be faked).
- Compares the monotonic-anchored trusted time against the system clock + offset.
- If drift exceeds 30 seconds, sends a msgTamper alert to the supervisor.

3. Why This Feature Is Needed

If Argus trusts the student's OS clock, the student controls the timestamps in all supervision logs. Clock manipulation could:

- **Conceal evidence** — shift clock backward so a suspicious connection appears to have occurred "before exam time".
- **Confuse the supervisor timeline** — mis-ordered events undermine incident analysis.
- **Bypass time-based checks** — heartbeat intervals, exam-duration windows, and timeout logic may behave incorrectly if client time is fake.
- **Destabilize the agent** — sudden time jumps can break timers and cause missed detections.

Reliable timestamps are essential evidence in a high-stakes exam environment. Cristian's Algorithm + monotonic drift detection closes this attack vector.

4. Algorithm & Formulas

Cristian's Algorithm Variables

Let T0 = time request sent, T3 = response received, ServerTime = supervisor's current UTC time.

RTT = T3 − T0	(round-trip time, ms)
Latency = RTT / 2	(estimated one-way delay)
TrustedTime = ServerTime + (Latency / 1000)	(seconds)
ClockOffset = TrustedTime − LocalSystemTime	
ActualTime = time(0) + ClockOffset	(all Argus timestamps)

Tamper Detection Workflow

```
(Startup)
[Student connects + secure handshake]
            |
            v
[Send msgTimeRequest × 3]
            |
            v
[Measure RTT per request / response]
            |
            v
[Select lowest RTT attempt]
            |
            v
[Compute ClockOffset, save serverSyncTime + monotonicSyncTime]
            |
            v
[All Argus timestamps: time(0) + clockOffset]

(Runtime — continuous)
[steady_clock::now() - monotonicSyncTime = elapsedSecs]
            |
            v
[trustedTime = serverSyncTime + elapsedSecs]
            |
            v
[systemTrusted = time(0) + clockOffset]
            |
            v
[drift = abs(trustedTime - systemTrusted)]
            |
            v
[drift > 30s  -->  send msgTamper to supervisor]
```

5. Codebase Implementation

A) Message Types — protocol.h

```
#define msgTimeRequest   6
#define msgTimeResponse  7
#define msgTamper        /* tamper alert type */
```

B) Supervisor Time Response — echo_supervisor.cpp

```
case msgTimeRequest: {
    time_t serverTime = time(0);
    string timeStr    = to_string(serverTime);
    Message response  = CreateMsg(msgTimeResponse, msg.studentID,
                                  serverTime, 0,
                                  timeStr.c_str(), timeStr.length());
    char respBuffer[1024];
    int  respSize = serialize(response, respBuffer,
                              clientContext[sd].sessionKey);
    send(sd, respBuffer, respSize, 0);
    break;
}
```

C) Clock Synchronization — echo_student.cpp · synchronizeClock()

```
for (int i = 0; i < 3; i++) {
    // send msgTimeRequest
    auto t0  = steady_clock::now();
    // recv msgTimeResponse
    auto t3  = steady_clock::now();
    long long rtt = duration_cast<milliseconds>(t3 - t0).count();
    // track bestRTT, bestServerTime, bestT3
}

if (success) {
    long long latency       = bestRTT / 2;
    time_t predictedTime    = bestServerTime + (latency / 1000);
    clockOffset             = predictedTime - time(0);
    serverSyncTime          = predictedTime;
    monotonicSyncTime       = bestT3;
    isTimeSynced            = true;
}
```

D) Trusted Timestamps for Outgoing Messages

```
msg.timestamp = isTimeSynced ? time(0) + clockOffset
                             : time(0);
```

E) Clock Tamper Detection — checkTampering()

```
void checkTampering() {
    if (!isTimeSynced) return;
    auto nowMono   = steady_clock::now();
    long long elapsed = duration_cast<seconds>(
                            nowMono - monotonicSyncTime).count();
    time_t trustedTime      = serverSyncTime + elapsed;
    time_t systemTrusted    = time(0) + clockOffset;
    long long drift         = abs(trustedTime - systemTrusted);
    if (drift > 30) {
        string alertMsg = "System Time Mismatch (Drift: "
                        + to_string(drift) + "s)";
        // Create and send msgTamper to supervisor
    }
}
```

SUMMARY *Argus anchors student timestamps to supervisor time using Cristian's Algorithm, then uses an unforgeable monotonic clock to detect OS time manipulation. Any drift over 30 seconds triggers an immediate tamper alert — preventing students from using clock manipulation to distort supervision logs.*

Feature 5 · **Huffman Coding Compression**

Lossless Compression for 512-byte Protocol Payloads

Files	huffman.h · huffman.cpp · echo_student.cpp · protocol.cpp · echo_supervisor.cpp
Algorithm	Huffman Coding (lossless prefix-free compression)
Constraint	Message.data fixed at 512 bytes (struct Message in protocol.h)

1. Algorithm Overview

Huffman Coding is a lossless compression algorithm that assigns short binary codes to frequently occurring characters and longer codes to rare ones, so the encoded output is shorter than the original for typical text inputs. Because codes are prefix-free (no code is a prefix of any other), decoding is always unambiguous.

2. What the Feature Does

A built-in compression layer for violation strings — such as unauthorized domains, SNI alerts, and long URLs. When the Student Agent needs to send a violation message:

- It attempts to compress the text using Huffman Coding.
- If the compressed result is smaller than the original, it sends msgViolationCompressed (type 20).
- Otherwise it sends a regular msgViolation.
- The receiver automatically decompresses inside deserialize(), so the rest of the system always works with readable plain text.

3. Why This Feature Is Needed

Argus uses a fixed-size message format: Message.data[512] cannot exceed 512 bytes. Long domain names, URLs, or detailed alert strings can exhaust or overflow this limit. Huffman compression:

- Packs more text into the 512-byte payload — reducing truncation risk.
- Keeps the protocol simple — no message fragmentation needed.
- Reduces bandwidth usage on constrained exam networks.

4. Algorithm Logic

Huffman Build Process

- Count byte frequencies across the input text.
- Push each (byte, frequency) pair into a min-heap priority queue.
- Repeatedly pop the two lowest-frequency nodes, merge them into a parent with combined frequency, push parent back.
- Continue until one root node remains — this is the Huffman tree.
- Traverse the tree: left edge = 0, right edge = 1; each leaf's path is its unique bit code.
- Encode: replace each character with its bit code, pack bits into bytes.

Decode: walk the tree bit-by-bit; when a leaf is reached, emit the character and reset to root.

Compression Workflow in Argus

```
[Student detects violation text (domain / URL / alert message)]
                |
                v
[Huffman Compress(text) --> compressedBuffer, compressedLen]
                |
                v
[compressedLen < originalLen ?]
     |                          |
     YES                        NO
     v                          v
[Send msgViolationCompressed]  [Send msgViolation (plain)]
     |                          |
     v                          v
            [Receiver deserialize()]
                |
                v
    [Decrypt payload (RC4)]
                |
                v
    [If msgViolationCompressed --> Decompress]
                |
                v
    [Supervisor receives plain-text msgViolation]
```

▪ 5. Codebase Implementation

A) Static Frequency Table — huffman.cpp · Init()

Argus uses a predefined frequency table optimized for English text and URL characters. Both Student and Supervisor initialize from this same table — meaning no tree metadata needs to be transmitted with each message.

```
void HuffmanCoding::Init() {
    frequencies[' '] = 20;  frequencies['e'] = 15;
    frequencies['t'] = 12;  frequencies['.'] = 15;
    frequencies['/'] = 10;  frequencies[':'] = 5;
    frequencies['w'] = 5;   // ... other common chars ...
    // Assign frequency 1 to all remaining bytes
    for (int i = 0; i < 256; i++)
        if (!frequencies.count((char)i))
            frequencies[(char)i] = 1;
    buildTree();
}
```

B) Tree Construction — buildTree() (min-heap)

```
void HuffmanCoding::buildTree() {
    priority_queue<HuffmanNode*, vector<HuffmanNode*>, Compare> pq;
    for (auto const& [ch, freq] : frequencies)
        pq.push(new HuffmanNode(ch, freq));

    while (pq.size() != 1) {
        HuffmanNode* left  = pq.top(); pq.pop();
        HuffmanNode* right = pq.top(); pq.pop();
        HuffmanNode* top   = new HuffmanNode('\0', left->freq + right->freq);
        top->left  = left;
        top->right = right;
        pq.push(top);
    }
    root = pq.top();
    generateCodes(root, "");
}
```

C) Compression — Compress() (bit packing)

```
void HuffmanCoding::Compress(
        const char* input, int inputLen, char* output, int& outputLen) {
    string bitStream = "";
    for (int i = 0; i < inputLen; i++)
        bitStream += huffmanCodes[input[i]];  // char --> bit string

    outputLen = 0;
    unsigned char currentByte = 0;
    int bitIndex = 0;
    for (char bit : bitStream) {
        if (bit == '1') currentByte |= (1 << (7 - bitIndex));
        if (++bitIndex == 8) {
            output[outputLen++] = currentByte;
            currentByte = 0; bitIndex = 0;
        }
    }
    if (bitIndex > 0) output[outputLen++] = currentByte;
}
```

D) Decompression — Decompress() (tree walk)

```
void HuffmanCoding::Decompress(
        const char* input, int inputLen, char* output, int& outputLen) {
    outputLen = 0;
    HuffmanNode* curr = root;
    for (int i = 0; i < inputLen; i++) {
        unsigned char byte = input[i];
        for (int b = 0; b < 8; b++) {
            bool isOne = (byte >> (7 - b)) & 1;
            curr = isOne ? curr->right : curr->left;
            if (!curr->left && !curr->right) { // leaf reached
                output[outputLen++] = curr->data;
                curr = root;
```

```
                if (outputLen >= 511) return;
            }
        }
    }
    output[outputLen] = '\0';
}
```

E) Transparent Integration — protocol.cpp · deserialize()

Decompression is seamless: the caller never needs to know compression was used.

```
// After RC4 decryption:
SecureDecrypt(msg->data, dataSize, key);

if (msg->msgType == msgViolationCompressed) {
    char decompressedData[512];
    int  newLen = 0;
    huffmanCoder.Decompress(msg->data, dataSize, decompressedData, newLen);
    memset(msg->data, 0, 512);
    memcpy(msg->data, decompressedData, newLen);
    msg->dataLength = newLen;
    msg->msgType    = msgViolation;  // rewrite to normal type
}
```

SUMMARY *Argus uses Huffman Coding to fit long violation strings into its strict 512-byte payload limit. The Student Agent compresses when beneficial and sends msgViolationCompressed; the receiver decompresses transparently inside deserialize() so the supervisor always receives readable plain-text violation data.*